1. **Цель работы**

   получение умений и навыков манипулирования данными в документных базах данных на примере MongoDB.

2. **Задание**

   1)Привести пример обновления и удаления данных из документной базы данных, разработанной в работе 9. (по 2 примера: один из вложенного документа, другой из документа коллекции верхнего уровня)

   2)Выполнить запросы на выборку из базы данных в соответствии с вариантом задания .(Запрос на поиск подстроки.

   Запрос на использование одного документа несколько раз.

   Запрос с условием на агрегатную функцию.

   Запрос получением агрегатной функции от агрегатной функции.

   Запрос на разность 2 запросов)

   Согласно варианту №6:

   поликлиника с учетом различного медицинского персонала):прием, пациент, процедура/прием, врач, стоимость, аппаратура. На приеме может быть только врач а процедуры могут вести и медсестры

   а. пациенты, приходившие на любые процедуры, , аппаратура для которой начинается со слова «лазер» , но это не единственная часть слова

   б. кабинет, где в один день проходят приемы нескольких врачей у одного пациента

   в. аппараты процедур с наибольшей стоимостью

   г. сотрудник поликлиники, работавший с наибольшим количеством различных пациентов в текущем году.

   д. пациент, не ходивший на процедуры к медсестре Ивановой

3. **Физическая модель**

   Разработанная физическая модель показана на рисунке 1.

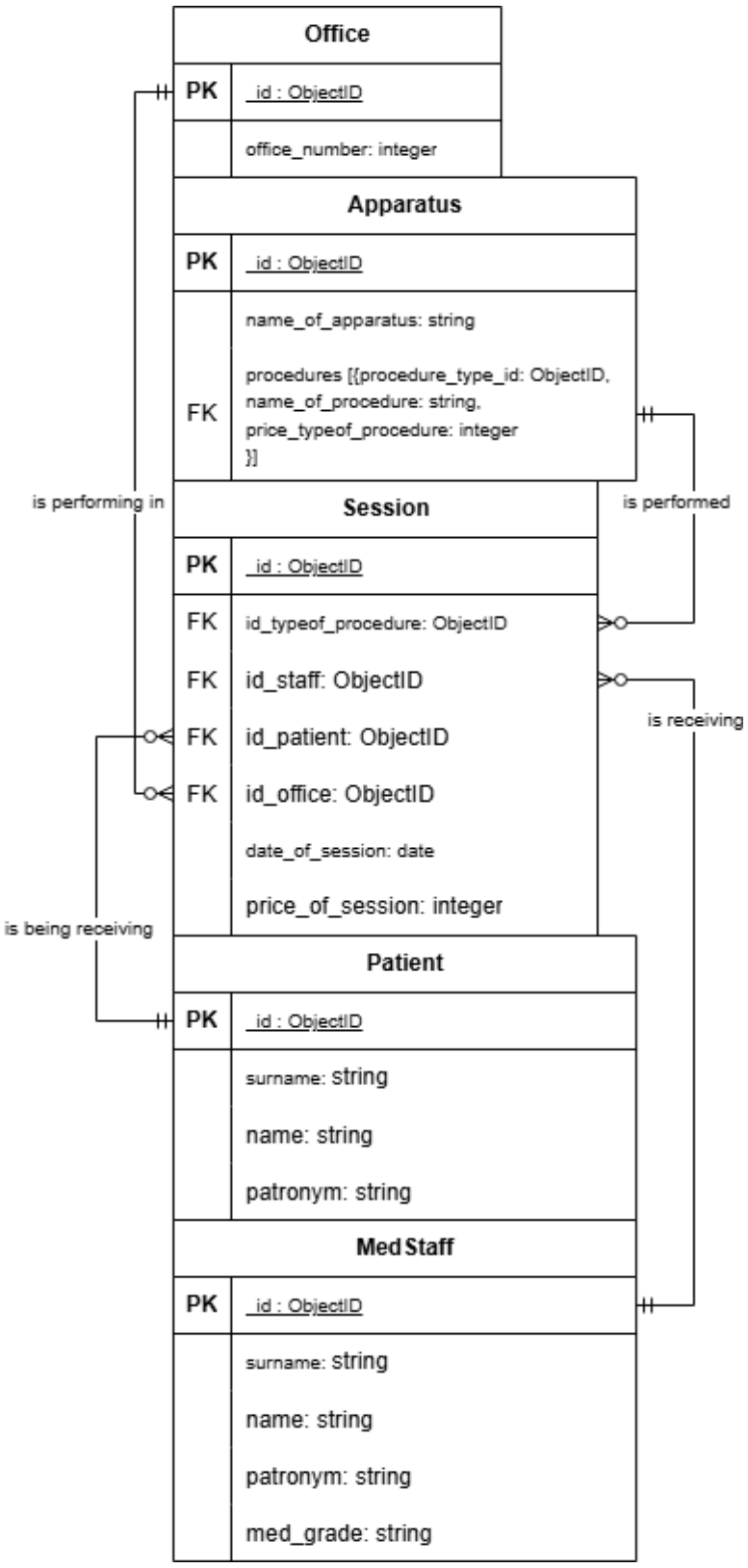


Рисунок 1 – Физическая модель

4. Таблица тестовых данных

В таблице 1 представлены тестовые данные для запросов.

Текст запроса	данные удовлетворяющие условиям запросов	данные не удовлетворяющие условиям запросов
пациенты, приходившие на любые процедуры, , аппаратура для которой начинается со слова «лазер» , но это не единственная часть слова	Петров	Все остальные
кабинет, где в один день проходят приемы нескольких врачей у одного пациента	Кабинет 305	Все остальные
аппараты процедур с наибольшей стоимостью	Лазерный офтальмоскоп МРТ	Все остальные
сотрудник поликлиники, работавший с наибольшим количеством различных пациентов в текущем году	Хаус Грегори Джеймс	Все остальные
пациент, не ходивший на процедуры к медсестре Ивановой	Иванов Петров Сидоров	Кузнецов

5. **Примеры обновления и удаления**

Обновление документа верхнего уровня

```
db.MedStaff.updateOne(
  { name: "Мария", patronym: "Петровна", med_grade: "nurse" },
  { $set: { surname: "Сидорова" } }
)
```

Обновление вложенного документа

```
db.Apparatus.updateOne(
  { "procedures.name_of_procedure": "МРТ спинного мозга" },
  { $set: { "procedures.$.price_typeof_procedure": 30000 } }
)
```

Удаления в документе верхнего уровня

```
db.Session.deleteMany({
date_of_session: {$gte: new Date("2026-01-01")}
```

```
})
Обновление вложенного документа
db.Apparatus.updateOne(
  { model: "МРТ" },
  { $pull: { procedures: { name_of_procedure: "МРТ головного мозга" } } }
)
```

6. Код запросов

a. пациенты, приходившие на любые процедуры, , аппаратура для которой начинается
со слова «лазер» , но это не единственная часть слова

```
db.Session.aggregate([
  // Присоединяем аппарат по типу процедуры
  { $lookup: {
      from: "Apparatus",
      localField: "id_typeof_procedure",
      foreignField: "procedures.procedure_type_id",
      as: "apparatus_info"
    }},
  // Оставляем только лазер
  { $match: {
      "apparatus_info.model":  /^Лазер.+/
    }},
  // Привязываем пациента к сессии
  { $lookup: {
      from: "Patient",
      localField: "id_patient",
      foreignField: "_id",
      as: "patient_info"
    }},
  { $unwind: "$patient_info" },
  { $unwind: "$apparatus_info" },
  { $project: {
    _id: 0,
```

```
    pat_id: "$patient_info._id",
    surname: "$patient_info.surname",
    name: "$patient_info.name",
    patronym: "$patient_info.patronym",
    apparatus_model: "$apparatus_info.model"
   }}
])
```

б.кабинет, где в один день проходят приемы нескольких врачей у одного пациента

```
db.Office.aggregate([
  {$lookup: {
     from: "Session",
     localField: "_id",
     foreignField: "id_office",
     as: "session_info"
    }},

   { $unwind: "$session_info" },

   {
    $match: {
      "session_info.id_typeof_procedure": null
    }
   },

   {
    $group: {
      _id: {
        office_id: "$_id",
        date: "$office_sessions.date_of_session",
        patient_id: "$office_sessions.id_patient",
        office_number: "$office_number"
      },
      uniq_doctors: { $addToSet: "$session_info.id_staff" }
```

```
    }
   },

   {
    $match: {
     $expr: { $gt: [{ $size: "$uniq_doctors" }, 1] }
    }
   },

   {
    $project: {
     _id: 0,
     office_number: "$_id.office_number",
     date: "$_id.date",
     patient_id: "$_id.patient_id",
     doctors_count: { $size: "$uniq_doctors" },
     staff_ids: "$uniq_doctors"
    }
   }
])
```

в. аппараты процедур с наибольшей стоимостью

```
db.Apparatus.aggregate([
  // Разворачиваем процедуры для группировки
  { $unwind: "$procedures" },
  // Среди всех ищем  максимум и записываем все процедуры
  { $group: {
     _id: null,
     maxPrice: { $max: "$procedures.price_typeof_procedure" },
     allProcedures: { $push: "$$ROOT" }
    }},
  // Разворачиваем все процедуры для фильтра
  { $unwind: "$allProcedures" },
  // Ищем те процедуры, цены которых равны максимуму
  { $match: {
```

```
      $expr: {
        $eq: ["$allProcedures.procedures.price_typeof_procedure", "$maxPrice"]
      }
    }},
  // Выкидываем лишние поля
  { $project: {
      _id: 0,
      model: "$allProcedures.model",
      procedure_name: "$allProcedures.procedures.name_of_procedure",
      price: "$allProcedures.procedures.price_typeof_procedure"
    }}
])
```

г. сотрудник поликлиники, работавший с наибольшим количеством различных пациентов в текущем году.

```
db.Session.aggregate([
  // Берём сессии за этот год
  { $match: {
      date_of_session: {
        $gte: new Date(new Date().getFullYear(), 0, 1),
        $lt: new Date(new Date().getFullYear() + 1, 0, 1)
      }
    }},
  // Группируем по сотруднику и считаем количество уникальных пациентов
  { $group: {
      _id: "$id_staff",
      uniquePatients: { $addToSet: "$id_patient" }
    }},
  // Выводим сотрудника и количество уникальных пациентов
  { $project: {
      staffId: "$_id",
      patientCount: { $size: "$uniquePatients" }
    }},
```

```
 // Группируем и ищем максимум пациентов
 { $group: {
    _id: null,
    maxPatients: { $max: "$patientCount" },
    allStaff: { $push: "$$ROOT" }
  }},
 // Разворачиваем сотрудников и выводим только с максимумом
 { $unwind: "$allStaff" },
 { $match: {
    $expr: { $eq: ["$allStaff.patientCount", "$maxPatients"] }
  }},
 // Присоединяем сотрудника
 {$lookup: {
    from: "MedStaff",
    localField: "allStaff.staffId",
    foreignField: "_id",
    as: "staff_info"
  }},
 // Выкидываем лишние поля
 { $unwind: "$staff_info" },
 { $project: {
    _id: 0,
    staff_id: "$allStaff.staffId",
    surname: "$staff_info.surname",
    name: "$staff_info.name",
    patronym: "$staff_info.patronym",
    med_grade: "$staff_info.med_grade",
    total_patients: "$maxPatients"
  }}
 ])
```

д. пациент, не ходивший на процедуры к медсестре Ивановой

```
db.Patient.aggregate([
 { $lookup: {
```

```
    from: "Session",
    let: { p_id: "$_id" },
    pipeline: [
     {
      $match: {
       $expr: { $eq: ["$id_patient", "$$p_id"] }
      }
     },
     // Соединение с персоналом для проверки сотрудника
     {
      $lookup: {
       from: "MedStaff",
       localField: "id_staff",
       foreignField: "_id",
       as: "staff_info"
      }
     },
     { $unwind: "$staff_info" },
     // Только медсестра Иванова
     {
      $match: {
       "staff_info.surname": "Иванова",
       "staff_info.med_grade": "nurse"
      }
     }
    ],
    as: "ivanova_visits"
  } },
 { $match: {
    ivanova_visits: { $size: 0 }
  } },
 { $project: {
    _id: 1,
    surname: 1,
    name: 1,
```

```
    patronym: 1
  } }
])
```

7. **Скриншоты выполнения запросов**

Результаты запросов а-д представлены на рисунках 2-6.

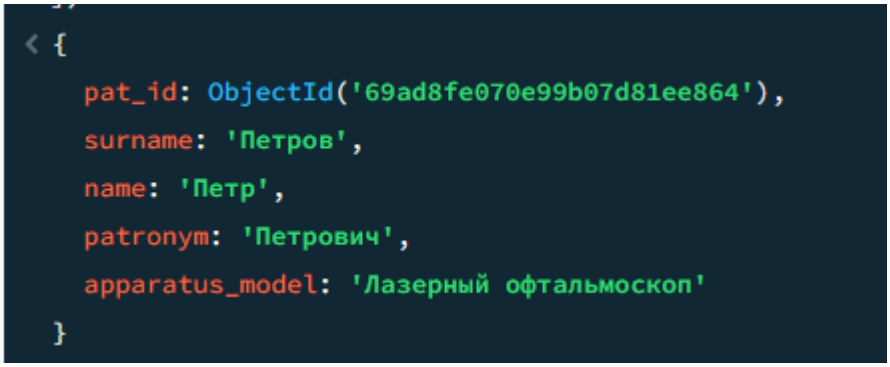


Рисунок 1 – Запрос а

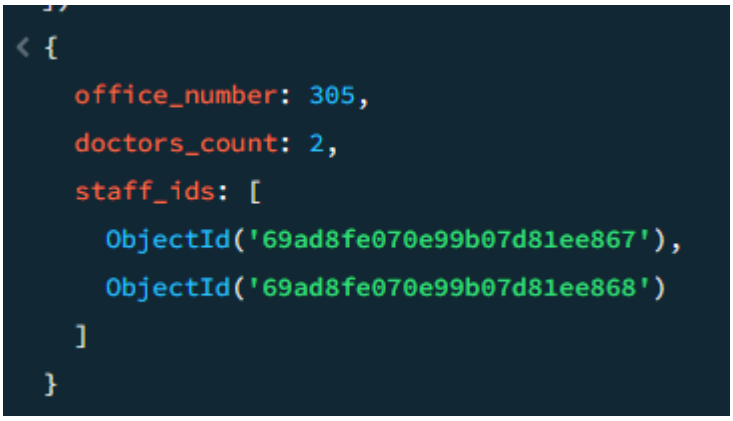


Рисунок 2 – Запрос б

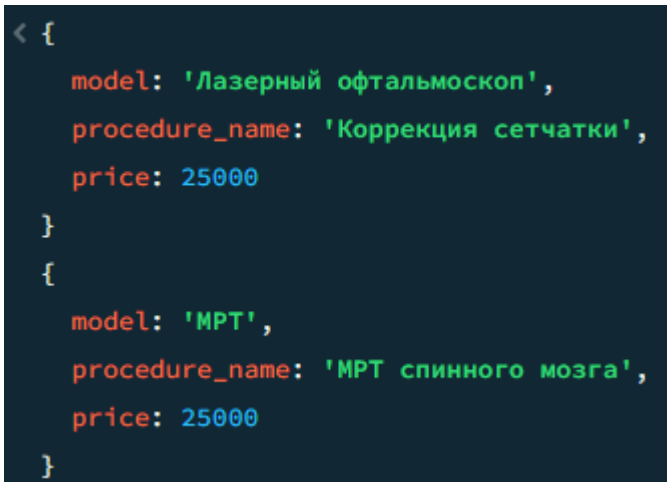


Рисунок 3 – Запрос в



Рисунок 4 – Запрос г



Рисунок 5 – Рисунок д

8. **Выводы**

В процессе лабораторной работы были получены практические навыки по составлению запросов в документной базе данных на примере MongoDB.